Lodz University of Technology
Faculty of Technical Physics, Information Technology and Applied Mathematics
Institute of Information Technology

Selected Issues of Network Applications
2026/2027

Modern Service Mesh: a better way to secure communication between microservices

Stage 2 report: Proof of Concept

Jędrzej Bartoszewski, 251482
Michał Domagała, 251505
Kacper Maziarz, 251586
Mikołaj Pawłoś, 258681

The problem definition, the choice of tools and the functional requirements are unchanged from the stage 1 report. This report covers what we built during the Proof of Concept stage and what we learned while doing it.

1. Work progress to date

#	Step	Planned	Status
1	k3d cluster and Online Boutique	Weeks 3–4	Done
2	Linkerd with mTLS (Proof of Concept)	Weeks 5–6, until 6.11.2026	Done
3	Zero-Trust authorization policies	Week 7	Next
4	Prometheus and Grafana	Week 8	Planned
5	Attack simulation	Week 9, until 27.11.2026	Planned

Table 1: Schedule from the stage 1 report and the current status.

We met the 6.11 deadline for both Proof of Concept steps, and the plan for steps 3–5 stays as it was.

Step 1: cluster and test application

The whole environment comes up with one script: `setup.sh` on Linux and macOS, `setup.ps1` on Windows. Docker is the only thing a team member has to install. The script downloads pinned versions of k3d (v5.9.0), kubectl (v1.37.2) and the Linkerd CLI (edge-26.11.1) into a local `.tools/` directory, so it needs no admin rights and ignores whatever versions happen to be installed system-wide. It then creates a k3d cluster named `sina` with one server and two agent nodes (`cluster/k3d-config.yaml`), deploys Online Boutique v0.10.7 from the manifest kept in the repo (`app/kubernetes-manifests.yaml`) and waits up to ten minutes for every deployment to become available. Host port 8080 is mapped to the k3d load balancer, which reaches the shop's `frontend-external` service, so the shop is at `http://localhost:8080` with no port-forward running.

Re-running the script reuses the existing cluster, which means anyone can run it again after a failed attempt without cleaning up first. `cluster/README.md` lists the failures we expect people to hit: pods stuck in `Pending` or `OOMKilled` when Docker has too little memory, Docker Desktop without WSL2 integration on Windows, and Podman in place of Docker on Linux.

Before we added the mesh, a fresh run had the cluster ready after 27 seconds, and all 12 pods reached `Running` less than a minute later, each with a single container (`1/1`). Figure 1 shows the running shop.

Step 2: Linkerd and mutual TLS

We installed Linkerd edge-26.11.1 with its CLI, and the steps live in `setup.sh` and `setup.ps1`, so the mesh comes up with the rest of the environment. The scripts install the Gateway API CRDs that Linkerd needs, run `linkerd install -crds` and `linkerd`

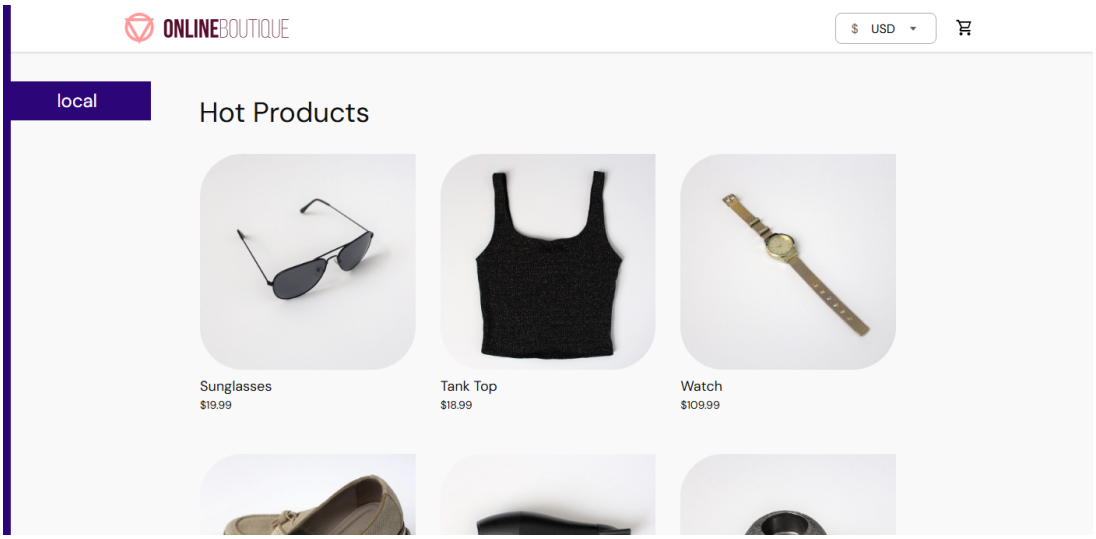


Figure 1: Online Boutique running on the local k3d cluster after `setup.sh`, reached through the k3d load balancer on `localhost:8080`.

`install`, confirm the result with `linkerd check`, and deploy the shop through `linkerd inject -native-sidecar`. That command adds the `linkerd.io/inject: enabled` annotation to each pod template as the manifest is applied. Every Online Boutique pod then starts with a second container, the `linkerd-proxy` sidecar, which takes over the pod's inbound and outbound traffic. The application code stayed untouched, and the manifest in the repo is the unmodified release file.

The `linkerd-viz` extension used for the checks below is not part of the setup scripts yet; we installed it separately with `linkerd viz install`. `linkerd check`, `linkerd check -proxy` and `linkerd viz check` all passed, and all 12 pods showed `2/2` containers.

We checked the result three ways.

1. `linkerd viz edges` lists every connection between deployments and whether it's secured (Figure 2). All 16 connections between application services showed as secured, and so did the 12 connections from the `linkerd-viz` Prometheus to those services.
2. `linkerd viz tap` streams individual live requests. Requests from `checkoutservice` to `paymentservice` carry `tls=true` (Figure 3): all 20 captured lines did.
3. A packet capture of `paymentservice` traffic, taken once with the service out of the mesh and once inside it (Figure 4). We placed five test orders during each capture. Without the proxy, the test card number and the currency code were readable in the packets of every `Charge` call. With the proxy, neither string appeared anywhere in the capture, and the traffic ran between the proxies on port 4143 as unreadable data.

The third check is the one we'll reuse in the final demo, because it shows the difference to anyone, Kubernetes background or not.

```
$ linkerd viz edges deploy -n default | grep -v prometheus
SRC                   DST                   SRC_NS    DST_NS    SECURED
cartservice           redis-cart            default   default   √
checkoutservice       cartservice           default   default   √
checkoutservice       currencyservice       default   default   √
checkoutservice       emailservice          default   default   √
checkoutservice       paymentservice        default   default   √
checkoutservice       productcatalogservice default   default   √
checkoutservice       shippingservice       default   default   √
frontend              adservice             default   default   √
frontend              cartservice           default   default   √
frontend              checkoutservice       default   default   √
frontend              currencyservice       default   default   √
frontend              productcatalogservice default   default   √
frontend              recommendationservice default   default   √
frontend              shippingservice       default   default   √
loadgenerator         frontend              default   default   √
recommendationservice productcatalogservice default   default   √
```

Figure 2: Connections between Online Boutique deployments as reported by `linkerd viz edges`. The rows for the `linkerd-viz` Prometheus are left out; they are secured as well.

```
$ linkerd viz tap deploy/checkoutservice --to deploy/paymentservice
req id=0:0 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
rsp id=0:0 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
end id=0:0 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true g...
req id=0:1 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
rsp id=0:1 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
end id=0:1 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true g...
req id=0:2 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
rsp id=0:2 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true :...
end id=0:2 proxy=out src=10.42.2.13:37546 dst=10.42.1.15:50051 tls=true g...
```

Figure 3: Live requests from `checkoutservice` to `paymentservice`, each marked `tls=true`.

```
$ tcpdump -A -s0 <payment pod traffic>

# outside the mesh (1/1), port 50051
20:02:34.002844 cni0  Out IP 10.42.2.13.38366 > 1...
E..m+.@.>..H
*.
*......
...:~?....@.......
.A....=...0..........+
.USD..........
.4432801561520454...... .
```

```
$ tcpdump -A -s0 <payment pod traffic>

# inside the mesh (2/2), port 4143
20:05:00.123754 veth34f3e223 Out IP 10.42.2.13.33...
E.....@.@...
*.
*...R./....6.z....I.......
..../.F{....L..V`X`.L.Z...+1....Q.m..XeC4...#T.V....
$jQ...5U_...<.3..M..oV.b.8..P/6.
```

Figure 4: One packet from each capture of `paymentservice` traffic, taken from the node, without (top) and with (bottom) the Linkerd proxy. The top packet carries the test card number in the clear.

2. Insights and challenges

Encryption isn't access control

This is the most important thing we learned in the PoC stage, and it changes how we present the project. After step 2, every connection between meshed services runs over mutual TLS, and each proxy knows the identity of its caller, derived from the caller's Kubernetes ServiceAccount (Online Boutique gives each of its 11 services its own). That stops eavesdropping and impersonation.

It doesn't stop lateral movement. Linkerd's default inbound policy is `all-unauthenticated`, so a meshed service still accepts requests from any other meshed service and even from a pod outside the mesh speaking plaintext. An attacker who controls `adservice` can't read the traffic between checkout and payments anymore, but can still call `paymentservice` directly and get an answer. We confirmed this with two test pods, one outside the mesh and one inside it running under the `adservice` ServiceAccount. Both reached the application: a `Charge` call with an empty body came back with the payment service's own error about the missing card, and `GetCart` on `cartservice` succeeded. Closing that gap is exactly what step 3 is for, and the schedule already has it in the right order; we just hadn't appreciated how big that gap would look on a slide until we had mTLS running.

Linkerd and Traefik's Gateway API CRDs

k3s installs Traefik by default, and its CRD chart clashes with the Gateway API CRDs that Linkerd needs: the Traefik install job kept crashing on a CRD ownership conflict. Nothing in the project uses Traefik, because the shop is reached through the k3d load balancer, so we disabled it in `cluster/k3d-config.yaml`.

Proving encryption took more effort than enabling it

Turning on mTLS took a few commands. Showing it to someone else took longer. Online Boutique images are distroless, with no shell and no tcpdump, and the pods set `runAsNonRoot: true`, which blocks the usual trick of attaching a root debug container. We captured from the node instead, using `kubectl debug node/... -profile=sysadmin`.

That route has a second advantage. Inside a meshed pod, the hop from the application to its own proxy goes over loopback and stays plaintext; a capture there would show readable traffic and look as if mTLS had failed. From the node you only see what leaves the pod, which is the traffic an attacker could reach.

Linkerd's release model

Since Linkerd 2.15 (February 2024), the open-source project publishes only edge releases, and stable builds come from Buoyant, the company behind Linkerd. We used edge release edge-26.11.1. For a student project that's fine, but edge releases can change behaviour between versions, so our setup scripts pin the version the same way they pin k3d and kubectl.

Resource cost

Each of the 12 application pods now runs an extra proxy container, plus the Linkerd control plane and the viz extension. In one `kubectl top` snapshot, shortly after the pods started, the 12 proxies used 43 MiB together, about 3.6 MiB each, next to 333 MiB for the application containers. The control plane used 120 MiB and the viz extension 264 MiB.

The main problem on a laptop was disk space rather than memory. When the system drive filled up on one of our Windows machines, Docker Desktop stopped responding while the cluster kept running, so `docker info` hung and the setup script hung with it. Restarting every deployment at once also strained the machine: `emailservice` and `recommendationservice` failed their health probes during start-up and restarted a few times before settling.

3. Impact on the next steps

Step 3 (Michał). We'll switch the namespace to a default-deny inbound policy and then allow only the calls Online Boutique actually makes, written as Linkerd `Server`, `AuthorizationPolicy` and `MeshTLSAuthentication` resources against the per-service identities. `paymentservice`, for example, will accept only the `checkoutservice` identity. Two things have to keep working: the frontend receives traffic from outside the mesh, and the loadgenerator must still reach it.

Step 4 (Kacper). `linkerd-viz` comes with its own Prometheus, but that instance keeps data for only six hours, and Linkerd no longer bundles Grafana. We'll start with the bundled Prometheus and add Grafana on top, because it's already running and needs less on our laptops. If it limits us, we'll move to a separate Prometheus that scrapes the proxies. Either way, the goal is one Grafana dashboard where the step 5 attack shows up as denied requests.

Step 5 (Michał). We'll run the same attack twice, once from a pod outside the mesh and once from inside a compromised `adservice`, both calling `paymentservice`. Before step 3 both calls succeed; afterwards both should fail with gRPC `PermissionDenied`.